

Documentation du déploiement

Ce document regroupe la **documentation du déploiement** de l'application **Hublot** : procédure d'installation, procédure de rollback, description de l'architecture, capture du pipeline CI et consultation des logs de build.

1. Procédure d'installation

Prérequis

- **Node.js** 20 LTS (ou compatible 18+)
- **npm** (livré avec Node.js)
- Compte **GitHub** et accès au dépôt
- Compte **Netlify** (pour le déploiement en ligne)

Installation en local

```
# Cloner le dépôt
git clone https://github.com/Meriem1403/Hublot.git
cd Hublot

# Installer les dépendances
npm ci

# (Optionnel) Variables d'environnement pour l'authentification
# Créer un fichier .env à la racine (jamais commité) :
# VITE_APP_USERNAME=admin
# VITE_APP_PASSWORD=votre_mot_de_passe

# Lancer en développement
npm run dev
```

L'application est alors disponible sur <http://localhost:5173>.

Vérification avant déploiement

```
# Exécuter les tests
npm run test:run

# Vérifier le build de production
npm run build
```

Le dossier `build/` est généré ; c'est ce dossier que Netlify publie.

Déploiement sur Netlify

1. Connexion du dépôt

- Sur app.netlify.com, Add new site → Import an existing project.
- Choisir **GitHub** et autoriser l'accès au dépôt **Meriem1403/Hublot**.
- Branche à déployer : **main**.

2. Paramètres de build (souvent détectés automatiquement via `netlify.toml` , voir **Annexe 02**) :

- **Build command** : `npm run build`
- **Publish directory** : `build`

3. Variables d'environnement (optionnel mais recommandé)

Dans Site settings → Environment variables :

- `VITE_APP_USERNAME` : identifiant de connexion
- `VITE_APP_PASSWORD` : mot de passe de connexion
- `NETLIFY_DATABASE_URL` : si utilisation de l'extension Neon pour la base de données

4. Déploiement

- Chaque **push sur** `main` déclenche automatiquement un nouveau build et un déploiement.
- L'URL du site est fournie par Netlify (ex. <https://dirmhublot.netlify.app>).

2. Procédure de rollback

En cas de problème après un déploiement, deux approches sont possibles.

Option A : Rollback depuis Netlify (sans toucher au code)

1. Se connecter à app.netlify.com et ouvrir le site.
2. Aller dans l'onglet **Deploys**.
3. Trouver un **déploiement précédent** dont le statut est Published et qui fonctionnait correctement.
4. Cliquer sur `...` (options) puis **Publish deploy** (ou Restore this version selon l'interface).

5. Ce déploiement redevient la version en ligne ; le code dans Git n'est pas modifié.

Option B : Rollback via Git (revenir à un commit antérieur)

1. Identifier le commit à restaurer :

```
git log --oneline
```

2. Revenir à ce commit (ex. `abc1234`) :

```
git revert --no-commit abc1234..HEAD
git commit -m "Rollback: retour à la version stable"
git push origin main
```

Ou, si l'équipe accepte de réécrire l'historique sur `main` (à utiliser avec précaution) :

```
git reset --hard abc1234
git push --force origin main
```

3. Netlify détecte le nouveau push et déploie la version correspondante.

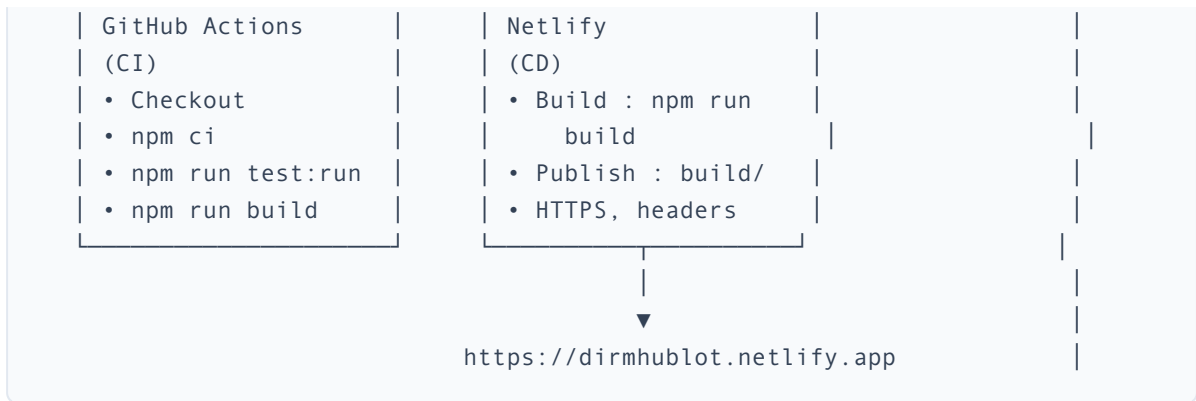
Recommandation : privilégier l'**option A** pour un retour rapide en production ; utiliser l'**option B** lorsque l'on souhaite aligner le dépôt et la version déployée.

3. Description de l'architecture

L'architecture de déploiement repose sur **Git** (source), **GitHub Actions** (CI) et **Netlify** (CD et hébergement).

Schéma simplifié





Fichiers de configuration

Annexe	Fichier	Rôle
Annexe 01	<code>.github/workflows/ci.yml</code>	Pipeline CI : lint, tests, audit, build, E2E (push/PR <code>main</code> et <code>staging</code>).
Annexe 02	<code>netlify.toml</code>	Commande de build, dossier à publier, redirections SPA, headers de sécurité.
Annexe 03	<code>package.json</code>	Scripts <code>build</code> , <code>test:run</code> , dépendances du projet.

Pour une description détaillée (dépôt, workflow YAML, build et déploiement automatiques), voir **Annexe 05** ([Annexes/Annexe_05_ARCHITECTURE_DEPLOIEMENT.md](#)).

4. Capture du pipeline CI

Le pipeline CI est défini dans **Annexe 01** (`.github/workflows/ci.yml`) et s'exécute sur **GitHub Actions**. Le CD Netlify est décrit dans [1.1.5 La mise en place de la livraison ou déploiement continu (CD)](/1.1 Les bases de la démarche DevOps/1.1.5 La mise en place de la livraison ou déploiement continu (CD).md).

Où voir le pipeline

1. Ouvrir le dépôt sur GitHub : github.com/Meriem1403/Hublot.
2. Onglet **Actions**.
3. Cliquer sur un **workflow run** (workflow **CI**) pour voir le détail des jobs et des étapes.

Étapes du pipeline (à capturer / décrire)

Étape	Nom dans le workflow	Action
1	Checkout	Récupération du code du dépôt
2	Setup Node.js	Installation de Node.js 20, cache npm
3	Install dependencies	<code>npm ci</code>
4	ESLint	<code>npm run lint</code>
5	Tests	<code>npm run test:run</code> (48 tests Vitest)
6	Audit prod	<code>npm run audit:prod</code>
7	Build	<code>npm run build</code>
8	E2E	<code>npm run test:e2e</code> (Playwright)

Comment capturer une « capture » du pipeline

- **Capture d'écran** : dans l'onglet Actions, ouvrir un run réussi et faire une capture de la liste des étapes (job build avec les coches vertes).
- **Export texte** : copier la sortie des logs d'une étape (bouton View job summary ou Download log archive).
- **Référence document** : « CI : workflow CI (lint, 48 tests, audit prod, build, E2E). CD : Netlify après required checks (`netlify.toml` , publish `build/`). Voir [1.1.5 La mise en place de la livraison ou déploiement continu (CD)](.. / 1.1 Les bases de la démarche DevOps / 1.1.5 La mise en place de la livraison ou déploiement continu (CD).md). »

Contenu type à inclure dans une capture ou une description :

- Nom du workflow : **CI** (jobs : ESLint, Tests unitaires, Audit npm, Build, E2E)
- Déclencheur : **push** et **pull_request** sur **main**
- Environnement : **ubuntu-latest**
- Liste des 5 étapes ci-dessus avec statut (succès / échec)

5. Logs de build

Les logs de build sont disponibles à deux endroits selon l'environnement.

Logs GitHub Actions (CI)

- **Emplacement** : GitHub → dépôt **Meriem1403/Hublot** → onglet **Actions** → cliquer sur un run → job **build**.
- **Contenu** : logs de chaque étape (Checkout, Setup Node.js, `npm ci`, `npm run test:run`, `npm run build`). En cas d'échec, l'étape en erreur affiche la sortie complète (tests, compilation, etc.).
- **Téléchargement** : possibilité de télécharger les logs en archive depuis l'interface du run.

Logs Netlify (build et déploiement)

- **Emplacement** : app.netlify.com → site **dirmhublot** (ou nom du site) → onglet **Deploys** → cliquer sur un déploiement.
- **Contenu** : logs du build Netlify (commande `npm run build`, sortie Vite, éventuelles erreurs). En cas d'échec, le message d'erreur et la stack sont visibles dans ces logs.
- **Filtres** : on peut filtrer par branche (ex. main) et par statut (Failed / Published).

Résumé

Source	URL / accès	Usage
GitHub Actions	Dépôt → Actions → workflow CI	Lint, tests, audit, build, E2E.
Netlify	Netlify Dashboard → Deploys → déploiement	Vérifier le build et le déploiement en production.

Pour un rapport ou une démonstration, on peut joindre un extrait des logs (ex. fin de l'étape Build avec `✓ built in ...`) ou une capture d'écran des deux interfaces.

Annexes

Tous les fichiers mentionnés dans cette documentation sont regroupés et numérotés dans le dossier **Annexes**. Index : [Annexes/README.md](#).

Annexe	Document / fichier	Description
	Annexe_01_build.yml	

Annexe	Document / fichier	Description
Annexe 01		Workflow CI (GitHub Actions)
Annexe 02	Annexe_02_netlify.toml	Configuration Netlify (build, headers, redirects)
Annexe 03	Annexe_03_package.json	Scripts et dépendances (extrait)
Annexe 04	Annexe_04_gitignore	Fichiers exclus du dépôt (.gitignore)
Annexe 05	Annexe_05_ARCHITECTURE_DEPLOIEMENT.md	Architecture détaillée (dépôt, workflow YAML, build, déploiement)
Annexe 06	Annexe_06_ENVIRONNEMENT_TEST.md	Environnements DEV, TEST, PROD
Annexe 07	Annexe_07_SECURITE_4_DEPLOIEMENT.md	Sécurité (HTTPS, variables d'env, headers, npm audit)
Annexe 08	Annexe_08_PLAN_TEST.md	Plan de test (tableau, scénarios, statuts)
Annexe 09	Annexe_09_DEMO_EPREUVE.md	Procédure d'exécution des tests (commandes)

Annexe	Document / fichier	Description
Annexe 10	Annexe_10_COMPETENCE_DEPLOIEMENT_STUDI.md	Validation compétence Studi
Annexe 11	Annexe_11_dataService.test.ts	Tests unitaires dataService (12 tests)
Annexe 12	Annexe_12_dataCalculations.test.ts	Tests unitaires dataCalculations (20 tests)
Annexe 13	Annexe_13_SCENARIOS_TEST.md	Scénarios de test formalisés (ST-F*, ST-SEC*, ST-AUTO*)